

Week 2 Day 2 — Raw Python vs. LangChain Agent

Comparison write-up and annotated reasoning trace

What changed going from raw Python to LangChain

Aspect	Day 1 — Raw Python	Day 2 — LangChain
Tool calling	Hand-written TOOL_CALL:/INPUT: text format, parsed with regex	@tool decorator; model's function-calling handles parsing
Agent loop	Manual for-loop with max_iterations guardrail	create_agent(...) compiles a LangGraph that runs reason/act/observe internally
Memory	scratchpad dict appended to messages list by hand	checkpoint=InMemorySaver() keyed by thread_id
Error handling	try/except inside execute_tool(), returns "ERROR: ..." string	@wrap_tool_call middleware catches exceptions, feeds them back as an Observation
Structured output	Not implemented — final answer is plain text	response_format=PydanticModel on create_agent returns a typed object
Visibility	Every prompt string and parsing step is visible and editable	Prompt construction and exception-to-message mapping are hidden inside the framework

Annotated reasoning trace

Query: "Look up the weather in Lahore and Tokyo and tell me which is warmer."

Step	Stage	What happened
1	Reason -> Act	Model decided to call get_weather(city="Lahore")
2	Observe	Tool returned "38C, Sunny"
3	Reason -> Act	Model decided to call get_weather(city="Tokyo")
4	Observe	Tool returned "27C, Clear"
5	Reason -> Final answer	Model compared 38C vs 27C and answered that Lahore is warmer

Tools implemented (Task 2)

Three tools, all built with the @tool decorator:

- calculator(expression) -- evaluates a basic arithmetic expression (reused from Day 1).
- get_weather(city) -- returns temperature and condition for a city (reused from Day 1).
- get_product_price(product_name) -- looks up price and specs for Laptop A (\$650), Laptop B (\$950), or Laptop C (\$1200) from a local JSON file, products.json (the new external data source for this task).

Each tool's docstring is sent to the model as part of the function-calling payload -- it is literally the text the model reads to decide when to call the tool and what argument to pass.

Task 4: Memory

Memory is implemented with checkpoint=InMemorySaver() passed to create_agent, keyed by a thread_id, and tested with the 3-turn scenario: "Find the price of Laptop A" -> "Now compare it to Laptop B" -> "Which one should I recommend to a budget-conscious client?" The third turn correctly referenced both laptops without re-stating their names, confirming the checkpointer persisted context across turns within the same thread_id.

Task 5: Structured Output & Error Handling

Structured output uses response_format=LaptopRecommendation (a Pydantic model with recommended_product, price_usd, and reason fields) passed directly to create_agent, returned in result["structured_response"].

Error handling uses a @wrap_tool_call middleware function (handle_tool_errors) registered via middleware=[...] on create_agent. get_product_price raises a ToolException for an unrecognized product name; the middleware catches

it and returns a ToolMessage with the error text instead of letting the exception crash the run, so the agent can reason over the failure and respond gracefully (e.g. explaining no such product exists).

What's similar vs. what's now hidden

- Similar: the underlying loop is still reason -> act -> observe -> repeat, and the same two tool calls happen in the same order as Day 1's raw log.
- Hidden now: Day 1 printed the literal prompt text sent to Gemini and the raw TOOL_CALL:/INPUT: string parsed with regex. `create_agent` hides that prompt construction, the tool-call parsing, and the loop's stop condition entirely inside the compiled LangGraph -- printing `result["messages"]` shows what was called and returned, but not the literal request/response text or the parsing logic underneath.
- Hidden now: the exact prompt LangChain assembles for function-calling, the internal LangGraph state machine `create_agent` compiles to, and how a raised Python exception gets mapped into the text the model actually reads as an Observation.

What LangChain made easier vs. what felt like "magic"

LangChain removed the need to hand-write the tool-call prompt format, the regex-based response parser, and the reason/act/observe loop itself -- `create_agent`, the `@tool` decorator, and `response_format` cover all of that, and `checkpoint=InMemorySaver()` replaced the hand-rolled scratchpad dict for multi-turn memory. The trade-off is reduced visibility: the exact prompt built for tool-calling, the internal LangGraph state machine underneath, and the exception-to-message mapping (handled via `@wrap_tool_call` middleware) happen inside the framework, so a wrong tool call is faster to fix here than in raw Python but harder to trace back to a root cause, since the intermediate text is no longer something the code prints or edits directly.